

```

"""
Supplement: Numerical Code for
"Topological Stiffness and Geometric Characterization of a 36-Node Kuramoto Ring"
Markus Drößler, MDV / Vermatrealize, Frankfurt am Main, 2026

Dependencies: numpy, scipy, matplotlib
Seed: numpy.random.seed(7) – all results are reproducible
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.optimize import linprog

# — Parameters —————
N = 36
np.random.seed(7)
omega_i = np.random.randn(N) # Gaussian frequencies, sigma=1 (normalized)

# — Core dynamics —————
def kuramoto_ring(t, theta, K, omega):
    """Nearest-neighbor Kuramoto ring."""
    ip = np.roll(theta, -1)
    im = np.roll(theta, 1)
    return omega + (K/2) * (np.sin(ip - theta) + np.sin(im - theta))

def order_parameter(theta):
    return np.abs(np.mean(np.exp(1j * theta)))

def get_steady_state(K, t_end=100):
    theta0 = np.random.uniform(0, 2*np.pi, N)
    sol = solve_ivp(kuramoto_ring, [0, t_end], theta0,
                    args=(K, omega_i), method='RK45', max_step=0.05)
    return sol.y[:, -1]

# — Ollivier-Ricci curvature —————
def phase_dist(a, b):
    d = abs(a - b) % (2 * np.pi)
    return min(d, 2 * np.pi - d)

def ollivier_ricci_edge(theta, i, alpha=0.5):
    """Discrete Ollivier-Ricci curvature of edge (i, i+1) via optimal transport."
    j = (i + 1) % N
    ip = (i - 1) % N

```

```

    jp = (j + 1) % N
    pos_i = [theta[ip], theta[i], theta[j]]
    pos_j = [theta[i], theta[j], theta[jp]]
    w = [(1 - alpha) / 2, alpha, (1 - alpha) / 2]
    C = np.array([[phase_dist(a, b) for b in pos_j] for a in pos_i])
    n = 3
    A = np.zeros((2 * n, n * n))
    for ii in range(n): A[ii, ii*n:(ii+1)*n] = 1
    for jj in range(n): A[n+jj, jj::n] = 1
    res = linprog(C.flatten(), A_eq=A, b_eq=np.tile(w, 2),
                  bounds=[(0, None)] * (n*n), method='highs')
    return 1 - (res.fun if res.success else np.nan)

def ricci_profile(theta, alpha=0.5):
    return np.array([ollivier_ricci_edge(theta, i, alpha) for i in range(N)])

# — Weighted graph Laplacian —————
def weighted_laplacian(theta, K):
    W = np.zeros((N, N))
    for i in range(N):
        ip = (i + 1) % N
        im = (i - 1) % N
        W[i, ip] += (K/2) * np.cos(theta[ip] - theta[i])
        W[i, im] += (K/2) * np.cos(theta[im] - theta[i])
    W = (W + W.T) / 2
    W = np.maximum(W, 0)
    return np.diag(W.sum(axis=1)) - W

def spectral_gap(L):
    return np.sort(np.linalg.eigvalsh(L))[1]

def dirichlet_energy(theta, K):
    E = 0
    for i in range(N):
        ip = (i + 1) % N
        w = (K/2) * abs(np.cos(theta[ip] - theta[i]))
        E += w * phase_dist(theta[i], theta[ip])**2
    return E / 2

# — Main sweep —————
if __name__ == "__main__":
    K_vals = np.linspace(0.5, 22, 60)
    results = {"K": K_vals, "r": [], "lambda2": [],
               "ricci_mean": [], "ricci_std": [], "dirichlet": []}

    print(f"Sweeping K over {len(K_vals)} values...")
    for K in K_vals:

```

```

theta = get_steady_state(K)
# Order parameter
results["r"].append(order_parameter(theta))
# Spectral gap
L = weighted_laplacian(theta, K)
results["lambda2"].append(spectral_gap(L))
# Ricci curvature
kappas = ricci_profile(theta)
results["ricci_mean"].append(np.nanmean(kappas))
results["ricci_std"].append(np.nanstd(kappas))
# Dirichlet energy
results["dirichlet"].append(dirichlet_energy(theta, K))

for k in results:
    results[k] = np.array(results[k])

# Key results
K_sat = K_vals[np.argmin(np.gradient(np.gradient(results["lambda2"], K_vals),
K_unif = K_vals[np.argmin(results["ricci_std"])]
print(f"\nSpectral saturation (lambda_2 inflection): K = {K_sat:.2f}")
print(f"Uniform geometry (min sigma_Ricci): K = {K_unif:.2f}")
print(f"lambda_2 at K=3.69: {np.interp(3.69, K_vals, results['lambda2']):.4f}")
print(f"lambda_2 at K=20.31: {np.interp(20.31, K_vals, results['lambda2']):.4f}")
print(f"sigma_Ricci at K=3.69: {np.interp(3.69, K_vals, results['ricci_std'])}")
print(f"sigma_Ricci at K=20.31: {np.interp(20.31, K_vals, results['ricci_std'])}")

# Summary plot
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, key, label, col in zip(
    axes,
    ["lambda2", "ricci_std", "dirichlet"],
    ["lambda_2 (Fiedler value)", "sigma(kappa_Ricci)", "Dirichlet energy"],
    ["blue", "red", "green"]
):
    ax.plot(K_vals, results[key], color=col, linewidth=2)
    ax.fill_betweenx(ax.get_ylim() if ax.get_ylim()[1] > 0 else [0,1],
                    3.0, 5.2, alpha=0.1, color='red')
    ax.axvline(K_sat, color='purple', linestyle=':', linewidth=1.5)
    ax.set_xlabel("K / sigma_omega"); ax.set_ylabel(label)
    ax.grid(True, alpha=0.3)
plt.suptitle("N=36 Kuramoto Ring: Geometric Measures", fontweight='bold')
plt.tight_layout()
plt.savefig("supplement_figure.png", dpi=150, bbox_inches='tight')
print("\nFigure saved: supplement_figure.png")

```